LLM Fine-Tuning: A Complete Guide

Assumes you already know Python, basic math, PyTorch, and ML fundamentals. Starts from how LLMs work.

---

1. How LLMs Work

Tokenization

- Models read **tokens**, not words. Most use BPE-style subword tokenizers.
- Learn: vocabulary, special tokens (BOS/EOS/PAD), and **chat templates** (the format that wraps system/user/assistant turns).
- **Most common beginner bug:** training with a different chat template than the one used at inference.

Transformer architecture

- **Embeddings** turn tokens into vectors; **positional encoding** (RoPE in most modern models) adds order.
- **Self-attention** lets each token weigh every earlier token. Learn queries, keys, values, and multi-head attention.
- **Decoder-only** models (Llama, Qwen, Mistral, Gemma) predict the next token; this is what you'll fine-tune.
- Each block = attention + feed-forward network + normalization + residual connections.

Pretraining vs. fine-tuning

- **Pretraining:** learns language and world knowledge from trillions of tokens. Not something you do.
- **Fine-tuning:** adjusts an existing model's behavior with a small, targeted dataset.
- **Base model:** raw next-token predictor. **Instruct model:** already tuned to follow instructions.
- Start from an **instruct** model if you want to tweak style/format; from a **base** model if you want full control over behavior.

Inference basics

- **Temperature, top-p, top-k** control randomness. **Context window** limits input + output length.
- **KV cache** speeds up generation by reusing past computation.
- Use **greedy/low temperature** when evaluating, so results are comparable.

Check yourself: can you explain why a model trained on short examples may behave badly on long inputs?

---

2. Should You Fine-Tune at All?

Work down this ladder and stop at the first rung that solves your problem:

1. **Better prompting** (clear instructions, examples, structured output)
2. **RAG** (retrieval) when the model needs *facts* it doesn't have or that change often
3. **Fine-tuning** when you need consistent **style, format, tone, or behavior**, lower latency/cost via a smaller model, or a specialized skill

Fine-tuning is poor at: reliably injecting large amounts of new factual knowledge.

Define success first: write down the task, 50-200 test cases, and the metric. Without this you can't tell if fine-tuning helped.

---

3. Data Preparation (most of your quality comes from here)

Formats

- **Instruction:** `{"instruction": ..., "input": ..., "output": ...}`
- **Chat/messages:** `{"messages": [{"role": "user", ...}, {"role": "assistant", ...}]}` (preferred)
- **Preference pairs** (for DPO): `{"prompt": ..., "chosen": ..., "rejected": ...}`

Building the dataset

- **Sources:** real logs, expert-written examples, public datasets, or synthetic data from a stronger model.
- **Clean:** deduplicate, remove empty/truncated/contradictory examples, strip PII and secrets.
- **Synthetic data:** generate with a strong model, then **filter and spot-check**. Never train on unreviewed generations.
- **Size:** a few hundred to a few thousand *high-quality, diverse* examples is often enough. Quality beats quantity.
- **Diversity:** cover edge cases, refusals, short and long inputs, and the exact formats you'll see in production.
- **Split:** train / validation / test. Keep the test set untouched and free of near-duplicates from train (**leakage** inflates results).

Train on the right tokens

- Compute loss only on the **assistant's response**, not the prompt (completion-only loss / assistant-only masking).
- Make sure every example ends with the EOS token so the model learns to stop.

---

4. Fine-Tuning Methods

Method	What it updates	Memory	When to use
Full fine-tuning	All weights	Very high	Large budgets, big behavior changes
LoRA	Small low-rank adapter matrices	Low	Default choice
QLoRA	LoRA on a 4-bit quantized base	Lowest	Limited GPU (Colab, a single consumer GPU)

LoRA essentials

- Freezes the base model and trains small adapter matrices, so you save a tiny file instead of a full model.
- **Rank (r):** adapter capacity (8-64 is typical; start at 16).
- **Alpha:** scaling factor (commonly 1-2x rank).

- **Target modules:** which layers get adapters (attention projections `q_proj, k_proj, v_proj, o_proj`; often MLP layers too).
- **Dropout:** small regularization (0.0-0.1).

Training stages

1. **SFT (supervised fine-tuning):** learn from input -> ideal output pairs. Always the first step.
2. **Preference alignment (optional):**

- **RLHF:** train a reward model, then optimize with RL (PPO). Complex.
- **DPO:** optimizes directly on chosen/rejected pairs. Simpler and very popular.
- **GRPO and RL with verifiable rewards:** used for reasoning tasks (math, code) where answers can be checked automatically.

Quantization

- 4-bit (NF4) and 8-bit reduce memory with a small quality cost. QLoRA relies on 4-bit NF4 + bf16 compute.

---

5. Tools and Setup

- **Hugging Face stack:** `transformers`, `datasets`, `peft`, `trl`, `accelerate`, `bitsandbytes`
- **Faster/easier wrappers:** Unsloth (speed and memory), Axolotl and LLaMA-Factory (config-driven)
- **Tracking:** Weights & Biases or TensorBoard
- **Compute:** Colab/Kaggle for models up to ~3B; rented GPUs (RunPod, Lambda, Vast) for 7B+
- **Rough VRAM guide (QLoRA):** ~7-8B model fits in 16-24 GB with modest sequence lengths.

Minimal QLoRA SFT example

```
import torch
from datasets import load_dataset
from peft import LoraConfig
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from trl import SFTConfig, SFTTrainer

model_id = "Qwen/Qwen2.5-1.5B-Instruct"  # pick any small instruct model

bnb = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
)
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id, quantization_config=bnb, device_map="auto"
)

lora = LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
    task_type="CAUSAL_LM",
)

# train.jsonl: one {"messages": [...]} object per line
ds = load_dataset("json", data_files={"train": "train.jsonl", "eval": "eval.jsonl"})

args = SFTConfig(
    output_dir="out",
```

```
    num_train_epochs=2,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,
    learning_rate=2e-4,
    lr_scheduler_type="cosine",
    warmup_ratio=0.03,
    bf16=True,
    logging_steps=10,
    eval_strategy="steps",
    eval_steps=50,
    save_steps=50,
    max_length=1024,
    gradient_checkpointing=True,
)

trainer = SFTTrainer(
    model=model, args=args,
    train_dataset=ds["train"], eval_dataset=ds["eval"],
    peft_config=lora, processing_class=tokenizer,
)
trainer.train()
trainer.save_model("out/adapter")
```

Library APIs change between versions (especially TRL). Check the current docs if an argument name errors.

---

6. Training in Practice

Key hyperparameters

- **Learning rate:** LoRA/QLoRA ~1e-4 to 2e-4; full fine-tuning ~1e-5 to 2e-5.
- **Epochs:** 1-3. More often means overfitting on small datasets.
- **Effective batch size:** `per_device_batch × gradient_accumulation × GPUs` (16-32 is a common target).
- **Scheduler and warmup:** cosine or linear decay, ~3-10% warmup.
- **Sequence length:** set to what your data needs; longer = much more memory.

Memory savers

- bf16/fp16 mixed precision, gradient checkpointing, smaller batch + more accumulation, 4-bit quantization, shorter sequences.

Troubleshooting

Symptom	Likely cause
Loss doesn't drop	Learning rate too low/high, bad data format, labels masked wrongly
Loss near zero fast	Duplicate/leaked data, tiny dataset (overfitting)
Model rambles, never stops	Missing EOS token in training data
Weird or broken outputs	Chat template mismatch between training and inference
Forgets general abilities	Too many epochs/high LR, narrow data (**catastrophic forgetting**); lower LR/rank or mix in general data
Out-of-memory	Reduce batch/sequence length, enable checkpointing, use QLoRA

---

7. Evaluation

- **Training/validation loss:** a sanity check, not a quality measure. Watch for validation loss rising while training loss falls.
- **Task metrics:** accuracy, F1, exact match, JSON validity rate, pass@k for code.
- **LLM-as-judge:** use a strong model to score outputs against a rubric. Beware of position and length bias; randomize order.
- **Human review:** read at least 50 real outputs. Numbers hide failures you'll spot immediately.
- **Regression check:** run a few general benchmarks or sample prompts to confirm you haven't broken the model's general ability.
- **Always compare** against (a) the base model, (b) the base model with a good prompt, and (c) your previous run.
- Evaluate on the **held-out test set only once per candidate**, to avoid tuning to it.

---

8. Deployment

- **Merge adapters** into the base weights (`merge_and_unload()` in PEFT) or serve them separately (vLLM supports LoRA adapters).
- **Export and quantize:** GGUF for llama.cpp/Ollama; AWQ/GPTQ for GPU serving.
- **Serving options:** vLLM (high-throughput GPU), Ollama/llama.cpp (local/CPU), Hugging Face endpoints (managed).
- **Production concerns:** latency, cost per request, versioning of models and datasets, logging real inputs for the next training round, monitoring quality drift.
- **Safety:** test for prompt injection and harmful outputs, and don't ship models trained on private data you can't expose.

---

9. Projects (one per level)

1. **Beginner:** QLoRA-tune a 1-3B instruct model on a public instruction dataset in Colab. Goal: complete the loop end to end.
2. **Intermediate:** build your own dataset for a narrow task (support replies in your style, structured JSON extraction). Compare against prompting with a clear metric.
3. **Advanced:** add DPO on preference pairs, build an automated eval suite with LLM-as-judge, merge, quantize, and serve with vLLM or Ollama.
4. **Stretch:** try GRPO on a verifiable task (math or code) and write up what you learned.

---

10. Study Checklist

- [ ] Explain tokenization, attention, and chat templates
- [ ] Decide between prompting, RAG, and fine-tuning for a given problem
- [ ] Build and clean an instruction/chat dataset with a proper split
- [ ] Explain LoRA rank, alpha, and target modules
- [ ] Run a QLoRA SFT job and read its loss curves
- [ ] Diagnose a failed run using the troubleshooting table

- [ ] Build an eval set and compare against the base model
- [ ] Merge, quantize, and serve a fine-tuned model
- [ ] Describe what DPO and GRPO add on top of SFT

Core resources

- Hugging Face LLM Course: https://huggingface.co/learn/llm-course
- Karpathy, Neural Networks: Zero to Hero: https://www.youtube.com/playlist?list=PLAqhIrjkxbuWI23v9cThsA9GvCAUhRvKZ
- Docs for `transformers`, `peft`, `trl`, Unsloth, and Axolotl (the primary source whenever APIs change)